

Face Recognition System — Development Changelog

Project Overview

A modular AI-powered face recognition system built with Python, OpenCV, face_recognition, MySQL, and CustomTkinter.

Changelog

Phase 1 — Core Environment Setup

Completed

- Installed Python environments
 - Python 3.9
 - Python 3.11
 - Python 3.14
 - Installed major dependencies
 - OpenCV
 - face_recognition
 - mysql-connector-python
 - numpy
 - CustomTkinter
 - Configured webcam access using OpenCV.
 - Created initial face detection pipeline.
-

Phase 2 — Face Detection System

Completed

- Built face detection system using:
 - OpenCV
 - Custom FaceDetectionModule

Features

- Webcam face detection
- Bounding box rendering
- Confidence detection

Files

- `FaceDetectionModule.py`
 - `recognition.py`
-

Phase 3 — Face Recognition System

Completed

- Added facial encoding generation.
- Added face comparison system.
- Added known/unknown user detection.
- Added confidence comparison logic.

Features

- Real-time recognition
- Unknown face handling
- Encoding generation

Libraries Used

- `face_recognition`
 - `numpy`
-

Phase 4 — Database Integration

Completed

- Installed and configured WAMP server.
- Restored Apache and MySQL services.
- Connected Python to MySQL.
- Created database:
 - `face_recognition_db`

Created Tables

users

Column	Type
id	INT AUTO_INCREMENT
name	VARCHAR(100)
Encoding	LONGTEXT

Features

- Save registered users
- Store face encodings
- Load users dynamically from database

Files

- `recognition.py`
 - `register_module.py`
-

Phase 5 — Registration Module

Completed

- Created modular registration system.
- Added image capture support.
- Added database insertion logic.
- Added dynamic user loading.

Features

- Register new users
- Save face encodings to database
- Reload users without restarting application

Files

- `register_module.py`
-

Phase 6 — GitHub Backup & Version Control

Completed

- Initialized Git repository.
- Connected project to GitHub.
- Successfully pushed project online.

Repository

- GitHub repository connected successfully.
-

Phase 7 — UI System Initialization

Completed

- Installed CustomTkinter.
- Built first modular UI system.
- Created independent UI module.
- Linked UI module to main application.

Features

- Modern dark-themed window
- Buttons
- Status labels
- Modular UI architecture

Files

- `ui_module.py`
 - `testui.py`
-

Current System Architecture

```
Vision/  
|  
├─ recognition.py  
├─ FaceDetectionModule.py  
├─ register_module.py  
├─ ui_module.py  
├─ testui.py  
|
```

```
├── assets/
│   ├── sounds/
│   ├── icons/
│   └── images/
└── models/
```

Current Features

Working Features

- Face detection
 - Face recognition
 - MySQL database storage
 - Dynamic user loading
 - User registration
 - GitHub backup
 - Modular UI system
 - CustomTkinter interface
-

Planned Features

UI Upgrades

- Embedded webcam feed
- Recognition status panel
- User cards
- Animated transitions
- Database viewer

AI Features

- Natural voice synthesis
- Qwen TTS integration
- Voice greetings
- Spoken recognition announcements

Security Features

- File locking system
- Face-based authentication

- Secure folder unlocking

Advanced Features

- Multi-user recognition
 - Confidence threshold tuning
 - Recognition history logs
 - AI assistant integration
-

Development Notes

Important Decisions

- Main project Python version: Python 3.9
- UI framework: CustomTkinter
- Database system: MySQL (WAMP)
- Architecture style: Modular

Lessons Learned

- Python version consistency is critical.
 - WAMP issues were caused by service/runtime conflicts.
 - Modular architecture greatly improves scalability.
 - Database-driven recognition is more scalable than image-only systems.
-

Current Project Status

Status

ACTIVE DEVELOPMENT

Stability

Core systems operational.

Next Development Goal

Embed webcam feed into the CustomTkinter UI.